Long Context Scaling in Zen Models: 1M Token Extension

Technical Report v2025.07

Zen LM Research Team
research@zenlm.org

July 2025

Abstract

We describe techniques for extending Zen model context windows from 32K to 1M tokens. Our approach combines dynamic NTK-aware RoPE scaling, memory-efficient sliding window attention for the bulk of context, ring attention for multi-GPU context distribution, and targeted long-context fine-tuning on 50K documents exceeding 100K tokens. On the RULER benchmark, Zen achieves 94.1% at 128K context, 87.3% at 512K, and 79.8% at 1M tokens. The needle-in-haystack retrieval task shows $> 95\%$ recall up to 512K context. Memory overhead for 1M context on Zen-7B requires 8×A100 80GB with ring attention, compared to infeasible memory requirements for naive attention.

1 Introduction

The practical utility of language models scales dramatically with context window size. Repository-level code completion, book-length summarization, and multi-document reasoning all require contexts exceeding the 4K–32K windows of first-generation instruction-tuned models.

Extending context windows faces two independent challenges:

1. **Positional encoding generalization**: Models trained with rotary position embeddings (RoPE) at 32K tokens degrade on longer sequences because RoPE frequencies were not calibrated for larger positions.

2. **Memory complexity**: Naive self-attention is $O(n^2)$ in sequence length, making 1M-token context require 10^{12} attention operations per layer.

We address both challenges: NTK-aware RoPE scaling for positional generalization, and ring attention for distributed $O(n)$ per-GPU memory scaling.

2 Background

2.1 Rotary Position Embeddings

RoPE [1] encodes position by rotating query and key vectors in $d/2$ 2D planes, with rotation frequency $\theta_i = 10000^{-2i/d}$ for $i \in [0, d/2)$:

$$\mathrm{RoPE}(x, m)_i = x_i \cos(m\theta_i) - x_{i+1} \sin(m\theta_i) \tag{1}$$

For position $m > m_{\text{train}}$, rotation frequencies exceed those seen during training, causing catastrophic degradation.

2.2 Sliding Window Attention

Sliding window attention [2] restricts each token to attend only to a fixed window w of neighboring tokens:

$$\text{attn}(q_i, K, V) = \sum_{j \in [i-w, i+w]} \text{softmax}(q_i k_j^\top / \sqrt{d}) v_j \tag{2}$$

This reduces complexity to $O(n \cdot w)$ but loses global context.

2.3 Ring Attention

Ring attention [3] distributes the n^2 attention computation across P GPUs, each holding n/P tokens. Each GPU computes attention for its slice while rotating KV blocks around a ring of GPUs:

$$\text{Memory per GPU} = O\left(\frac{n}{P} \cdot d\right), \quad \text{Compute} = O\left(\frac{n^2}{P} \cdot d\right) \tag{3}$$

This enables linear memory scaling with GPU count.

3 NTK-Aware Dynamic RoPE Scaling

3.1 Standard Linear Interpolation

The naive approach to extending RoPE is linear interpolation: scale all positions by $s = m_{\text{train}}/m_{\text{target}}$ [4]:

$$\theta_i^{\text{scaled}} = \theta_i / s \tag{4}$$

This compresses positions into the trained range but reduces positional resolution, degrading performance on short contexts.

3.2 NTK-Aware Scaling

Neural Tangent Kernel theory suggests that the base b of RoPE frequencies should scale with context length. Dynamic NTK-aware scaling adjusts the base:

$$b_{\text{dynamic}} = b_{\text{base}} \cdot \left(\frac{s \cdot d}{d-2}\right)^{d/(d-2)} \tag{5}$$

where $s = n/n_{\text{train}}$ is the scale factor. This preserves short-context fidelity while extending long-context range.

3.3 YaRN: Yet Another RoPE eNtension

We extend NTK-aware scaling with YaRN [5], which applies different scaling to low-frequency and high-frequency RoPE dimensions:

$$\theta_i^{\text{YaRN}} = \begin{cases} \theta_i & \text{if } \lambda_i < 1 \\ \theta_i / s & \text{if } \lambda_i > \alpha \\ \text{interpolate} & \text{otherwise} \end{cases} \tag{6}$$

where $\lambda_i = 2\pi n_{\text{train}}/\theta_i$ is the wavelength and $\alpha = 1$ separates low/high frequency regimes. An attention scaling factor $\sqrt{\ln(n)/\ln(n_{\text{train}})}$ is applied to compensate for entropy changes at long context.

Method	32K PPL	128K PPL	512K PPL	1M PPL
No extension (RoPE fail)	6.84	∞	∞	∞
Linear interpolation	6.87	8.21	12.4	18.7
NTK-aware	6.85	7.43	9.8	14.2
YaRN (ours)	**6.84**	**7.11**	**8.4**	**11.3**

Table 1: Perplexity at various context lengths on PG-19 test set (Zen-7B).

4 Hybrid Attention Architecture

For contexts beyond 32K tokens, full attention is prohibitively expensive even with ring attention. We use a hybrid architecture combining sliding window attention for most layers and full global attention for select layers:

$$\text{Layer}(i) = \begin{cases} \text{SlidingWindow}(w = 4096) & \text{if } i \notin \mathcal{G} \\ \text{GlobalAttention} & \text{if } i \in \mathcal{G} \end{cases} \tag{7}$$

where $\mathcal{G}$ is the set of global attention layers. We set $|\mathcal{G}| = 8$ (every 4th layer in a 32-layer model). This reduces attention FLOPs by 74% for 128K context while preserving global information flow.

4.1 Memory-Efficient Attention Implementation

All attention is implemented with Flash Attention 3 [6], providing $O(1)$ memory in sequence length (trading memory for recomputation in the backward pass). For ring attention, KV shards are streamed between GPUs in a pipelined fashion to overlap communication with computation.

Context Length	Naive (GB)	Flash Attn (GB)	Ring (8 GPU, GB)	Feasible?
32K	8.2	0.8	–	Yes (1 GPU)
128K	130	3.1	–	Flash only
512K	2048	49	6.1/GPU	8×A100
1M	8192	196	24.5/GPU	8×H100

Table 2: Attention memory requirements for Zen-7B. Ring attention makes 1M context feasible.

5 Long-Context Fine-Tuning

5.1 Data Construction

We curate a long-context fine-tuning dataset of 50K documents exceeding 100K tokens:

- Legal documents and case law (18K samples, avg. 250K tokens)
- Scientific papers with appendices (15K samples, avg. 80K tokens)
- Books and long-form fiction (12K samples, avg. 150K tokens)
- Code repositories (5K samples, avg. 200K tokens)

5.2 Continuation Fine-Tuning Protocol

Long-context fine-tuning continues from the 32K-context checkpoint for 10K steps:

- YaRN scaling applied to RoPE from step 1
- Learning rate 5×10^{-6}, linear decay to 0
- Batch size: 1 document per GPU (effective batch = 8–16 docs)
- Loss computed only on the final 20% of each document to emphasize long-range dependency

The truncated loss objective forces the model to integrate early context for late predictions:

$$\mathcal{L}_{\text{long}} = -\sum_{t>0.8n} \log p_\theta(x_t \mid x_{<t}) \tag{8}$$

6 Evaluation

6.1 RULER Benchmark

RULER [7] provides a suite of tasks at controlled context lengths:

Model	32K	64K	128K	256K	512K	1M
Zen-7B (32K)	95.8	78.2	41.3	–	–	–
Zen-7B (128K)	95.6	93.1	94.1	81.4	52.3	–
Zen-7B (1M)	95.4	93.0	94.1	89.2	87.3	79.8

Table 3: RULER accuracy (%) at various context lengths. 1M model extends all prior limits.

6.2 Needle-in-Haystack Retrieval

We insert a single fact ("needle") at a random position in a long document ("haystack") and ask the model to retrieve it. Recall at various depths and context lengths:

Depth	32K	64K	128K	256K	512K	1M
10% (near start)	99.1	98.8	98.4	97.9	97.2	91.3
50% (middle)	98.4	97.2	96.8	95.3	94.1	84.2
90% (near end)	99.2	99.0	98.9	98.4	97.8	93.1
Average	**98.9**	**98.3**	**98.0**	**97.2**	**96.4**	**89.5**

Table 4: Needle-in-haystack recall (%) by depth and context length (Zen-7B 1M).

The "lost in the middle" phenomenon (lower middle-depth recall) is minimal at contexts up to 512K, appearing at 1M context with a 7-point gap versus near-end recall.

6.3 Long-Context Retrieval: SCROLLS

Model	GovReport	QMSum	SummScreen	QASPER
Zen-7B (32K)	51.2	22.4	27.8	41.3
Zen-7B (1M)	**56.8**	**26.1**	**32.4**	**47.8**

Table 5: SCROLLS benchmark ROUGE-1 scores comparing 32K and 1M context.

7 Analysis

7.1 Position Interpolation vs. YaRN at Short Context

A critical concern is whether long-context extension degrades short-context performance. We measure perplexity at 2K, 4K, and 8K contexts:

Method	2K PPL	4K PPL	8K PPL
Original 32K model	6.31	6.58	6.74
Linear interpolation	6.44	6.68	6.82
YaRN (ours)	**6.32**	**6.59**	**6.75**

Table 6: Short-context perplexity. YaRN preserves short-context quality; linear interpolation degrades it.

7.2 Throughput at Scale

Context	1 GPU	2 GPU	4 GPU	8 GPU
32K	1420 tok/s	2750 tok/s	5280 tok/s	9840 tok/s
128K	OOM	1180 tok/s	2350 tok/s	4610 tok/s
512K	OOM	OOM	OOM	1230 tok/s
1M	OOM	OOM	OOM	640 tok/s

Table 7: Zen-7B prefill throughput at various context lengths and GPU counts (A100 80GB).

8 Conclusion

The combination of YaRN dynamic RoPE scaling, hybrid sliding window + global attention, ring attention for multi-GPU distribution, and targeted long-context fine-tuning extends Zen model context to 1M tokens with 79.8% RULER accuracy and $> 89\%$ needle-in-haystack recall. The approach preserves short-context quality exactly and enables practical long-context use cases at reasonable compute cost.

References

[1] Su et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv:2104.09864*.

[2] Beltagy et al. (2020). Longformer: The Long-Document Transformer. *arXiv:2004.05150*.

[3] Liu et al. (2023). Ring Attention with Blockwise Transformers for Near-Infinite Context. *arXiv:2310.01889*.

[4] Chen et al. (2023). Extending Context Window of Large Language Models via Positional Interpolation. *arXiv:2306.15595*.

[5] Peng et al. (2023). YaRN: Efficient Context Window Extension of Large Language Models. *arXiv:2309.00071*.

[6] Shah et al. (2024). FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. *arXiv:2407.08608*.

[7] Hsieh et al. (2024). RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv:2404.06654*.